

The scalable section-domain framework

Procedural encoding, exact energy, association, and current evidence

Stage 4 of 4
Updated 30 August 2026
Status Jobs 023–027 audited; job 027 partial after four diagnostic failures

This report is a chronological research record. Measured results, interpretations, and open hypotheses are labelled separately. Detailed numerical ledgers remain in **results/**.

1 What changes at large payload

At $B = 100$, a global count vector has 2^{100} coordinates, so neither explicit storage nor an implicit matrix with a global decoder state is meaningful. The scalable boundary is crossed only by replacing the arbitrary global lookup T_ℓ with a procedure that maps payload bits directly to a short path of local indices.

Let section ℓ have alphabet $\mathcal{X}_\ell = \{0, \dots, N_\ell - 1\}$. The outer encoder is

$$f_{\text{out}} : \{0, 1\}^B \longrightarrow \mathcal{X}_1 \times \dots \times \mathcal{X}_L, \quad f_{\text{out}}(w) = (i_1(w), \dots, i_L(w)).$$

It executes once per payload; there is no table with 2^B rows. Each active user contributes one local atom per section. If $s_\ell \in \mathbb{Z}_+^{N_\ell}$ counts those choices, the receiver's linear state has total size $\sum_\ell N_\ell$ and

$$y = \sum_{\ell=1}^L F_\ell s_\ell, \quad F_\ell \in \mathbb{C}^{n \times N_\ell}$$

for overlapping banks, or the orthogonal construction below. This physical representation is M-free. Recovering the complete messages still requires associating local choices into valid paths.

2 Exact power for every procedural codeword

Sampled energy penalties cannot guarantee deployment-time power on unseen payloads. The current exact construction gives section ℓ a local bank $G_\ell \in \mathbb{C}^{n_\ell \times N_\ell}$ whose every column has unit norm. Choose energies $E_\ell > 0$ with $\sum_\ell E_\ell = 1$, concatenate disjoint latent subspaces, and apply a fixed orthogonal mixer $Q_{\text{mix}} \in \mathbb{C}^{n \times n}$:

$$\phi(w) = Q_{\text{mix}} \begin{bmatrix} \sqrt{E_1} G_1[:, i_1(w)] \\ \vdots \\ \sqrt{E_L} G_L[:, i_L(w)] \\ 0 \end{bmatrix}.$$

Orthogonality gives, for every valid or invalid local tuple,

$$\|\phi(w)\|_2^2 = \sum_{\ell=1}^L E_\ell \|G_\ell[:, i_\ell(w)]\|_2^2 = \sum_{\ell} E_\ell = 1.$$

The mixer spreads energy over physical channel uses without destroying the direct-sum proof. Projecting learned local columns back to unit norm after each update preserves the guarantee. Overlapping full-length banks remain available for experiments, but only a sampled penalty applies there because cross terms depend on the complete path.

3 Outer code and parity matrix

The implemented sparse-linear outer code uses symbols in the modular ring $\mathbb{Z}_q$ with $q = 2^J$. Calling this $\text{GF}(2^J)$ would be inaccurate for the present arithmetic: $\mathbb{Z}_{2^J}$ is not a field when $J > 1$. Payload bits are grouped into $k = B/J$ information symbols $u = (u_1, \dots, u_k) \in \mathbb{Z}_q^k$. The encoder adds r parity symbols and produces a path $x \in \mathbb{Z}_q^L$, $L = k + r$.

Each sparse check is

$$\sum_{\ell \in \mathcal{N}(c)} H_{c\ell} x_\ell = 0 \pmod{q}.$$

Stacking r checks produces

$$Hx = 0 \pmod{q}, \quad H \in \mathbb{Z}_q^{r \times L}.$$

For a systematic parity section p_c , the row contains coefficients on a few information symbols and coefficient one on p_c . The encoder computes that parity directly:

$$p_c = - \sum_{j \in S_c} h_{cj} u_j \pmod{q}.$$

Thus H is simply a compact list of which section symbols participate in each modular sum. It is section-sized, never message-sized.

In the current implementation, graph supports and coefficients in H are fixed hyperparameters. Learning them would be a discrete code-design problem; continuous relaxations are possible but would change the experiment and do not automatically preserve invertibility, sparsity, or good decoding. Local bank entries and decoder calibration are the continuous learned parameters.

Parity adds transmitted sections/channel resources but not new payload information. For example, the current CCS-like default has $B = 128$, $J = 16$, $k = 8$ information sections and $r = 8$ parity sections, so $L = 16$ local transmissions encode the same 2^{128} payloads. The redundancy is what rules out most cross-section combinations. One must not replace the message count 2^B by $L2^J$: a complete path is a joint tuple, so the unconstrained path space is $(2^J)^L = 2^{JL}$, and the parity checks select 2^B valid paths from it.

4 Local D0 and its gradients

For batched local states $s_\ell^t \in \mathbb{R}^{P \times N_\ell}$, define

$$r^t = y - \sum_{\ell} F_{\ell} s_{\ell}^t, \quad g_{\ell}^t = \Re\{F_{\ell}^H r^t\}, \quad u_{\ell}^t = s_{\ell}^t + \eta_t g_{\ell}^t.$$

Each active message selects one atom in every section, so $\mathbf{1}^T s_{\ell} = K_a$. Under uniform payloads, one coordinate has the exact marginal count prior

$$S_{\ell,j} \sim \text{Binomial}(K_a, 1/N_{\ell}).$$

The denoiser evaluates posterior weights for counts $c = 0, \dots, K_a$,

$$\pi(c | u) \propto \binom{K_a}{c} p^c (1-p)^{K_a-c} \exp\left[-\gamma_t(u-c)^2/(2\tau_t) + b_t c\right],$$

then uses $\mathbb{E}[S | u]$ as the proposal and $\log \Pr(S > 0 | u) - \log \Pr(S = 0 | u)$ as support evidence. There is no separate collision/no-collision branch.

For any loss whose upstream signal gradient is g_y , the physical encoder receives

$$\frac{\partial \mathcal{L}}{\partial s_{\ell}} = F_{\ell}^H g_y.$$

Local bank gradients follow from the same adjoint chain. The outer path is discrete, so ordinary backpropagation learns the selected column values and decoder parameters, not the integer symbol choices or H itself.

5 Why association is a separate problem

Suppose two users and two sections produce perfect local multisets

$$s_1 : \{a, b\}, \quad s_2 : \{c, d\}.$$

The complete messages could be (a, c) , (b, d) or (a, d) , (b, c) . Both give the same two section-count vectors and therefore the same y . This is not a decoder optimisation failure; without an outer constraint the complete messages are not identified by this representation.

Parity constraints reject many recombinations, but a multiuser receiver observes a *set* of valid paths. Single-user marginal beliefs can be individually plausible while their cross-section pairing is wrong. This distinction explains why better local support loss or better section marginals need not improve complete-message PUPE.

6 Outer BP and list extraction

The modular factor graph has variable nodes $x_\ell \in \mathbb{Z}_q$ and check nodes. D0 logits supply unary probabilities. A check-to-variable update sums products of neighbouring messages over assignments satisfying its modular equation:

$$m_{c \rightarrow \ell}(x_\ell) \propto \sum_{\{x_j : j \in \mathcal{N}(c) \setminus \ell\}} \mathbf{1} \left\{ \sum_{j \in \mathcal{N}(c)} H_{cj} x_j = 0 \pmod{q} \right\} \prod_{j \neq \ell} m_{j \rightarrow c}(x_j).$$

Variable updates multiply the unary evidence and other incoming check messages. The implementation performs these full-alphabet operations differentiably, with learned temperature, damping, and check gain.

Evaluation then uses a beam over systematic information symbols. Whenever a parity becomes determined, its belief is added to the path score; each completed information tuple is encoded to its unique valid path. For $B \leq 20$, local counts are fitted to candidate-path multiplicities. For larger payloads the evaluator assumes complete-message collisions are negligible and selects unique paths.

The discrete top- k /beam decisions are not differentiated. Training instead uses D0 count loss, outer marginal loss, and a contrastive objective that ranks every true path above random and cross-user recombination negatives. This can train evidence so the true path enters the beam, but it does not make the beam itself a smooth optimiser.

7 Controlled evidence from jobs 023–026

7.1 Execution and energy

Job 023 returned 16 of 26 small $B = 16$ rows. Exact-energy triadic banks were better than the overlapping sampled-energy row; the latter reached sampled codeword energy about 1.29. BP was worse than D0 in eight of nine returned aggregates and outer-loss gradient norms reached 10^{15} before clipping, so fine graph rankings are not trustworthy.

Job 024 completed every $B = 128, J = 16, n = 38400$ row. Its state was $16 \times 65,536 = 1,048,576$ local coordinates rather than 2^{128} ; maximum unit-energy error was 4.8×10^{-7} . However, PUPE was 1.0 everywhere, support loss stayed near 15, logits saturated at the lower clamp, and initial gradients were 10^{-14} – 10^{-13} . It certifies execution and power, not useful learning.

7.2 Causal bridges

Job 025 used $L = 1$ and the same dense, sparse-global, and ODMA physical codebooks as the earlier landscape. Global D0 and section-compatible D0 matched exactly in every layer logit, soft output, and hard output. Separately trained Binomial D0 differed by less than 0.0037 PUPE. The backend change and collision-aware prior are therefore not the source of the later gap.

Job 026 introduced $L > 1$ while retaining a small explicit comparator. Signal equivalence held within 6.0×10^{-7} and energy within 6.6×10^{-7} .

High-SNR PUPE	$K_a = 9$	$K_a = 17$	$K_a = 30$
Identity, local association	.9453	.9692	.9862
Triadic fixed, local association	.5191	.7909	.8945
Triadic learned, local	.4141	.8272	.9469
Triadic learned, BP	.4340	.8465	.9573

Table 1: Average over 8 and 12 dB. The identity row has no parity factor and is an identifiability control.

Triadic constraints clearly help at low load, but performance collapses as occupancy rises. With $J = 4$, each local alphabet has only 16 atoms, so local collisions are nearly certain even while global message collisions remain modest. BP worsened every reported high-SNR triadic cell by 0.007–0.022. Because the explicit comparator is D0 rather than exact ML/MAP, this is a practical current-system gap, not proof that every procedural encoder excludes a good solution.

8 Sparse-global density frontier

Job 027 returned all 72 array logs, but only 68 summaries/checkpoints. Four full-density seed-2702 runs completed training and evaluation and then failed in posthoc support diagnostics: a finite float32 Gaussian draw contained one exact zero, so one column had numerical support 255 rather than 256. Initialisation now resamples exact zeros, but these four rows must be rerun before the strict aggregate is complete. Every other artifact has the expected history and evaluation cells, finite values, paired D0/D1 codebooks, exact nested support/sign inclusion, and energy error below 9.54×10^{-7} .

The table averages PUPE over 8/12 dB and $K_a = 9, 17, 26, 30$. The dense and $s = 256$ entries use the completed four-decimal logs for the otherwise missing second seed and are therefore contextual rather than strict-summary values.

Codebook	$p = s/256$	D0	D1
Dense	1	.2935	.2239
Sparse global, $s = 256$	1	.2979	.2226
Sparse global, $s = 64$	1/4	.2932	.2256
Sparse global, $s = 48$	3/16	.2975	.2188
Sparse global, $s = 32$	1/8	.3051	.2276
Sparse global, $s = 16$	1/16	.3312	.2323
Sparse global, $s = 8$	1/32	.3984	.2813
Sparse global, $s = 4$	1/64	.5171	.3994
Four-mask ODMA	1/4	.5363	.4884

Table 2: Job 027 high-SNR density frontier. Lower PUPE is better.

D0 remains essentially flat through $s \approx 48$; D1 is within about .008 of dense even at $s = 16$, although its two-seed uncertainty and still-descending 20-epoch loss make $s = 8$ –24 a transition

band rather than a precise optimum. At equal density $p = 1/4$, random sparse global beats ODMA by .243 PUPE under D0 and .263 under D1. Even $s = 4$ remains better than ODMA. This confirms that nonzero fraction alone is not the structural issue.

The raw number of masks is also not the limiting event. All 4,096 support/sign patterns remain distinct through $s = 4$, yet degradation begins earlier. From $s = 32$ to $s = 8$, sampled correlation q99.9 rises from .256 to .479 and the fraction of rows occupied by a $K_a = 30$ active set falls from .982 to .613. At $s = 1$, only 256 supports and 512 real signed directions are available, producing 87.5% signed-pattern repetition and PUPE above .83.

This bank still stores a dense 256×4096 tensor with zeros and decodes an M -length state. It identifies a promising sparse model class; it does not implement the desired procedural $B = 100$ solution.

The resulting controlled next choices are:

1. Rerun only the four failed full-density rows, then add replication at $s = 8, 16, 32, 64$.
2. Confirm those few points at $B = 14, n = 256$ to distinguish an absolute-support from a density scaling rule.
3. Design or research a compact procedural support generator with controlled pair geometry, balanced rows, and an inverse/search method; compare both recovery and geometry against the explicit frontier.
4. Repair large- J D0 calibration using pseudo-data scale, clamp fraction, and top- K local recall gates before returning to $B = 128$.
5. Optimise and evaluate complete paths or message sets, with exact tiny-scale enumeration as a reference. Retain BP only if it first helps on tree, single-user, and controlled multiuser cases.

The research direction is therefore constrained but not predetermined: preserve the dense/sparse-global recovery lead, remove the 2^B state, guarantee power, and require the decoder to recover complete messages rather than only attractive section marginals.